

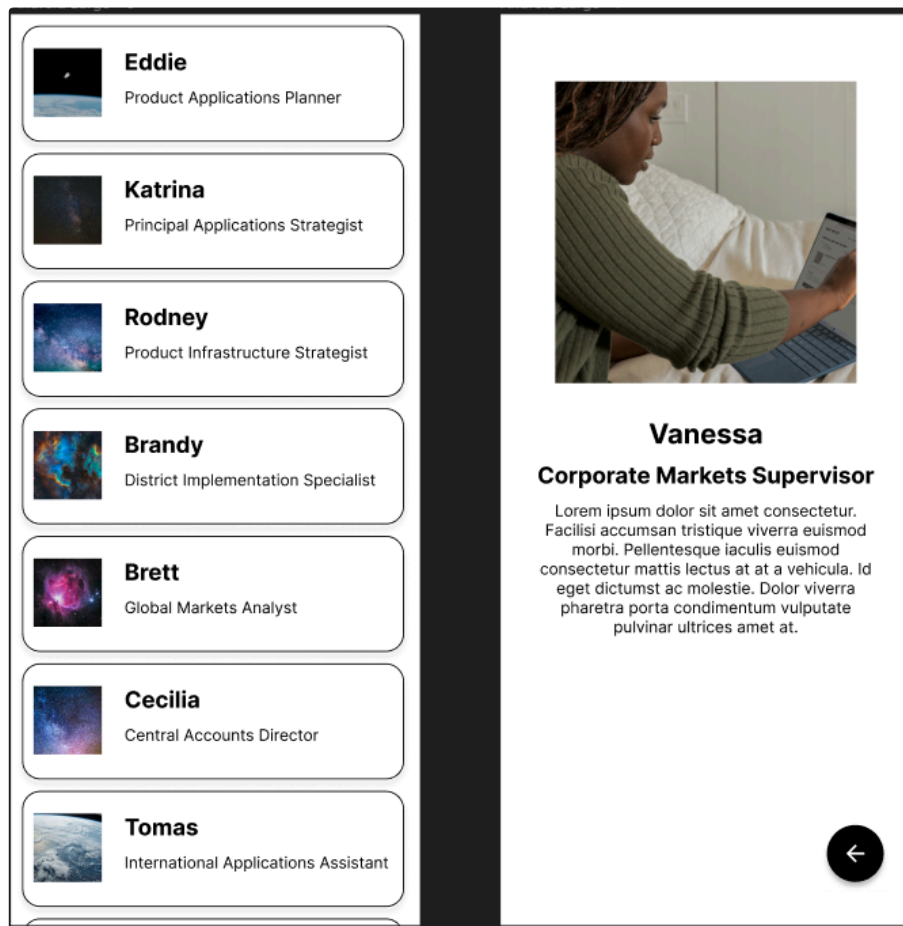
Exemple avec les ListView

Dans ce lab, nous allons voir ensemble comment implémenter un design dans une application Android en utilisant les `ListView`.

Rappel: une `ListView` est un composant de l'interface utilisateur qui permet d'afficher une liste défilable d'éléments.

Design

Pour le design, nous allons intégrer deux écrans. Le premier va lister des personnages avec leurs noms et titres, et lorsqu'on clique sur un élément de la liste on navigue vers un second écran pour voir plus de détails.



Implémentation

Pour l'implémentation, nous allons lister les membres de l'équipage du chapeau de paille de l'oeuvre One Piece.

La ListView

Le model

Nous allons créer une nouvelle classe Java qui représente les données d'un personnage. Nous aurons comme attribut:

- L'image

- Le nom
- Le titre
- La description

```
1 import java.io.Serializable;
2
3 public class MangaCharacter implements Serializable {
4     private String image;
5     private String name;
6     private String title;
7     private String description;
8
9     public MangaCharacter(String image, String name, String title, String description) {
10         this.image = image;
11         this.name = name;
12         this.title = title;
13         this.description = description;
14     }
15
16     public String getImage() {
17         return image;
18     }
19
20     public void setImage(String image) {
21         this.image = image;
22     }
23
24     public String getName() {
25         return name;
26     }
27
28     public void setName(String name) {
29         this.name = name;
30     }
31
32     public String getTitle() {
33         return title;
34     }
35
36     public void setTitle(String title) {
37         this.title = title;
38     }
39
40     public String getDescription() {
41         return description;
42     }
43
44     public void setDescription(String description) {
45         this.description = description;
46     }
47 }
```

Nous n'oublions pas de marquer la classe avec l'interface `Serializable` vu que nous allons passer les objets à une `Intent` plus tard lorsqu'on va créer la navigation entre la liste et l'écran qui représente les détails.

Le template

On va créer deux nouvelles layout, la première qui représente un élément de la liste, et la seconde pour ajouter la bordure sur la première.

1. Créer un premier fichier xml dans le répertoire drawable qu'on va nommer `border.xml` :

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <shape xmlns:android="http://schemas.android.com/apk/res/android" android:shape="rectangle">
3     <corners android:radius="16dp"/>
4     <padding android:left="10dp" android:right="10dp" android:top="10dp" android:bottom="10dp"/>
5     <stroke android:width="1dp" android:color="#000000"/>
6 </shape>
```

- Élément racine `<shape>` : Définit une forme pour être utilisée comme arrière-plan ou autre élément graphique dans une application Android.
- Attribut `xmlns:android` : Définit l'espace de noms pour les attributs spécifiques à Android, permettant l'utilisation de propriétés prédéfinies.
- Attribut `android:shape` : Spécifie la forme de l'élément graphique, ici un rectangle.
- Élément `<corners>` : Définit les propriétés des coins de la forme.
- Attribut `android:radius` : Arrondit les coins de la forme avec un rayon de 16dp (unité de mesure indépendante de la densité).
- Élément `<padding>` : Spécifie le remplissage à l'intérieur de la forme.
- Attributs `android:left`, `android:right`, `android:top`, `android:bottom` : Définit le remplissage à gauche, à droite, en haut et en bas respectivement à 10dp, ce qui ajoute un espace entre le contenu intérieur de la forme et ses bords.
- Élément `<stroke>` : Définit un contour pour la forme.
- Attribut `android:width` : Définit la largeur du contour à 1dp.
- Attribut `android:color` : Définit la couleur du contour, ici noir (`#000000`).

Nous allons utiliser ce code pour créer une forme de rectangle avec des coins arrondis, un contour noir, et un remplissage spécifié.

2. Créer un second fichier nommé `row.xml` :

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="match_parent"
4     android:layout_height="wrap_content"
5     android:paddingVertical="20dp"
6     android:paddingHorizontal="10dp"
7     android:background="@drawable/border"
8 >
9
10 <ImageView
11     android:id="@+id/pic"
12     android:layout_width="60dp"
13     android:layout_height="60dp"
14     android:src="@drawable/luffy"
15 />
16
17 <TextView
18     android:id="@+id/name"
19     android:layout_width="wrap_content"
20     android:layout_height="wrap_content"
21     android:textStyle="bold"
22     android:textSize="20sp"
23     android:layout_alignParentTop="true"
24     android:layout_toEndOf="@id/pic"
25     android:layout_marginStart="20dp"
26     android:text="Name" />
```

```

27
28     <TextView
29         android:id="@+id/title"
30         android:layout_width="wrap_content"
31         android:layout_height="wrap_content"
32         android:textSize="14sp"
33         android:layout_toEndOf="@id/pic"
34         android:layout_below="@id/name"
35         android:layout_marginStart="20dp"
36         android:layout_marginTop="10dp"
37         android:text="Title" />
38
39
40 </RelativeLayout>

```

Ce code contient une `RelativeLayout` qui elle même contient trois éléments:

- `ImageView` : Affiche une image.
- `TextView` pour le nom : Affiche du texte.
- `TextView` pour le titre : Affiche du texte.

L'Adapter

On va maintenant créer notre `Adapter` qui va itérer sur la liste des characters et retourner les vues en utilisant notre template précédemment définis.

On va maintenant créer une classe personnalisée pour un adaptateur Android, `MangaCharacterAdapter`, qui étend `ArrayAdapter<MangaCharacter>`. Cette classe est utilisée pour peupler des éléments la `ListView` avec un tableau de `MangaCharacter`

```

1  import android.content.Context;
2  import android.view.LayoutInflater;
3  import android.view.View;
4  import android.view.ViewGroup;
5  import android.widget.ArrayAdapter;
6  import android.widget.ImageView;
7  import android.widget.TextView;
8
9  import androidx.annotation.NonNull;
10 import androidx.annotation.Nullable;
11
12 public class MangaCharacterAdapter extends ArrayAdapter<MangaCharacter> {
13
14     Context context;
15     int layoutId;
16     MangaCharacter[] data;
17
18     public MangaCharacterAdapter(@NonNull Context context, int resource, @NonNull MangaCharacter[] objects) {
19         super(context, resource, objects);
20         this.context = context;
21         layoutId = resource;
22         data = objects;
23     }
24
25     @Override
26     public MangaCharacter getItem(int position) {
27         return super.getItem(position);
28     }
29
30     @NonNull

```

```

31     @Override
32     public View getView(int position, @Nullable View convertView, @NonNull ViewGroup parent) {
33
34         View row = convertView;
35
36         LayoutInflater inflater = LayoutInflater.from(context);
37         row = inflater.inflate(layoutId, parent, false);
38
39         ImageView pic = row.findViewById(R.id.pic);
40         TextView name = row.findViewById(R.id.name);
41         TextView title = row.findViewById(R.id.title);
42
43         MangaCharacter character = data[position];
44
45         name.setText(character.getName());
46         title.setText(character.getTitle());
47
48         int imageResId = context.getResources().getIdentifier(character.getImage(), "drawable", context.getPacka
49         pic.setImageResource(imageResId);
50
51         return row;
52     }
53 }

```

- `context` : Le contexte dans lequel l'adaptateur est utilisé (souvent une activity).
- `layoutId` : L'identifiant de la ressource de mise en page qui définit comment chaque élément de la liste doit être affiché.
- `data` : Un tableau des données (`MangaCharacter[]`), contenant les éléments à afficher dans la liste.
- Le constructeur initialise l'adaptateur avec le contexte, la ressource de mise en page, et les données:
 - `super(context, resource, objects)` : Appelle le constructeur de la classe parent (`ArrayAdapter`), passant le contexte, l'ID de ressource de mise en page, et les données.
- `getItem(int position)` : Retourne l'élément à la position spécifiée dans l'ensemble de données.
- La méthode `getView` est appelée pour chaque élément de la liste quand l'élément devient visible:
 - `LayoutInflater` : Utilisé pour gonfler la mise en page de l'élément de la liste à partir de l'ID de ressource spécifié.
 - Initialisation de la vue:
 - `row` : La vue de l'élément actuel. Si `convertView` est non-nul, il peut être réutilisé, sinon une nouvelle vue est gonflée.
 - `pic`, `name`, `title` : Les vues dans la mise en page de l'élément sont initialisées par leurs ID respectifs.
 - Configuration des données :
 - `MangaCharacter character` : L'objet `MangaCharacter` pour la position actuelle.
 - Les méthodes `setText` sur `name` et `title` sont utilisées pour définir le texte des vues `TextView`.
 - Ressource d'image : L'ID de la ressource d'image est déterminé et utilisé pour définir l'image dans `ImageView`.

Configurer la ListView

1. Ouvrir le fichier `activity_main.xml`, et le modifier pour y rajouter la `ListView` :

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      xmlns:tools="http://schemas.android.com/tools"
5      android:id="@+id/main"
6      android:layout_width="match_parent"
7      android:layout_height="match_parent"
8      tools:context=".MainActivity">
9

```

```

10     <ListView
11         android:id="@+id/characters_list"
12         android:layout_width="match_parent"
13         android:layout_height="match_parent"
14         android:dividerHeight="10dp"
15         android:divider="@color/white"
16         android:padding="10dp"/>
17
18 </RelativeLayout>

```

2. On va maintenant mettre à jour la classe `MainActivity`

```

1  import android.content.Intent;
2  import android.os.Bundle;
3  import android.view.View;
4  import android.widget.AdapterView;
5  import android.widget.ListView;
6
7  import androidx.appcompat.app.AppCompatActivity;
8
9  public class MainActivity extends AppCompatActivity {
10
11      MangaCharacter[] characters = new MangaCharacter[] {
12          new MangaCharacter("luffy", "Monkey D. Luffy", "Capitaine des Mugiwras", "Fondateur et capitaine de
13          new MangaCharacter("zoro", "Roronoa Zoro", "Vice capitaine des Mugiwras", "Vice capitaine de l'équip
14          new MangaCharacter("nami", "Nami", "Navigatrice des Mugiwras", "Nami alias \"La chatte voleuse\", es
15          new MangaCharacter("usopp", "Usopp", "Tireur d'élite des Mugiwras", "Tireur d'élite de l'équipage du
16          new MangaCharacter("sanji", "Vinsmoke Sanji", "Cuisinier des Mugiwras", "Surnommé \"Sanji la Jambe N
17          new MangaCharacter("chopper", "Tony Tony Chopper", "Medecin des Mugiwras", "Il est le medecin de l'é
18          new MangaCharacter("robin", "Nico Robin", "Archéologue des Mugiwras", "Aka \"l'Enfant du démon\", el
19          new MangaCharacter("franky", "Franky", "Charpentier des Mugiwras", "Franky est le charpentier de l'é
20          new MangaCharacter("brook", "Brook", "Musicien des Mugiwras", "Surnommé Brook le fredonneur est le m
21          new MangaCharacter("jimbe", "Jinbe", "Timonier des Mugiwras", "Jinbe \"Le Paladin des Mers\" est le
22      };
23      MangaCharacterAdapter mangaCharacterAdapter;
24      ListView listView;
25
26      @Override
27      protected void onCreate(Bundle savedInstanceState) {
28          super.onCreate(savedInstanceState);
29          setContentView(R.layout.activity_main);
30
31          listView = findViewById(R.id.characters_list);
32          mangaCharacterAdapter = new MangaCharacterAdapter(this, R.layout.row, characters);
33          listView.setAdapter(mangaCharacterAdapter);
34      }
35  }

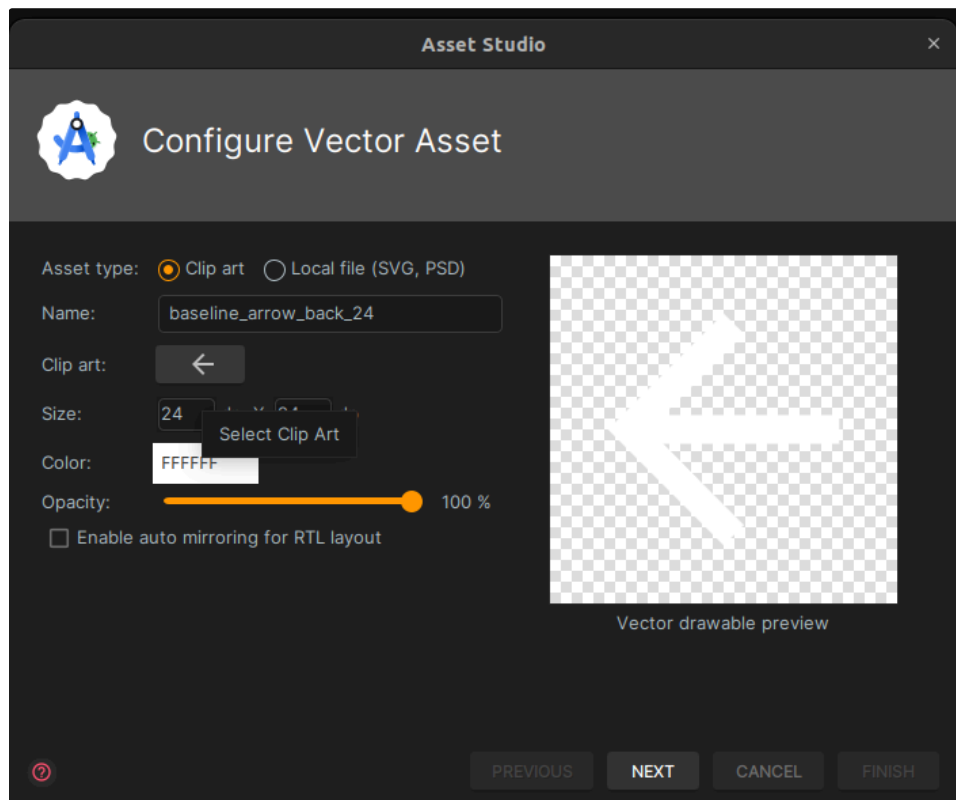
```

 Pour chaque `MangaCharacter`, il faut ajouter une image dans le répertoire `drawable`, vous pour trouver les images sur ce lien  [mugiwras - Google Drive](#)

Voir plus de détails

1. Créer une nouvelle activity la page qui affiche les détails d'un personnage, on le nommera `MangaCharacterActivity`
2. Ajouter une icone pour le FAB:

Faites un clique droit sur le projet, puis sur `New` et ensuite sur `Vector Asset`



Cliquez sur l'icone à droite sur la troisième ligne pour changer l'icone, et rechercher `baseline_arrow_back_24` pour avoir le même icone que sur l'image. N'oubliez pas de changer la couleur.

Normalement un nouveau fichier xml devrait apparaitre dans le répertoire drawable.

3. Mettre à jour la layout:

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      xmlns:tools="http://schemas.android.com/tools"
5      android:id="@+id/main"
6      android:layout_width="match_parent"
7      android:layout_height="match_parent"
8      android:paddingHorizontal="32dp"
9      android:paddingTop="32dp"
10     tools:context=".MangaCharacterActivity">
11
12     <RelativeLayout
13         android:layout_width="match_parent"
14         android:layout_height="match_parent">
15
16         <ImageView
17             android:id="@+id/pic"
18             android:layout_width="300dp"
19             android:layout_height="300dp"
20             android:src="@drawable/luffy"
21             android:layout_centerHorizontal="true" />
22
23         <TextView
24             android:id="@+id/name"
25             android:layout_width="wrap_content"

```

```

26         android:layout_height="wrap_content"
27         android:layout_centerHorizontal="true"
28         android:layout_marginTop="30dp"
29         android:textSize="24sp"
30         android:textStyle="bold"
31         android:layout_below="@id/pic"
32         android:text="Name" />
33
34     <TextView
35         android:id="@+id/title"
36         android:layout_width="wrap_content"
37         android:layout_height="wrap_content"
38         android:layout_centerHorizontal="true"
39         android:layout_marginTop="10dp"
40         android:textSize="20sp"
41         android:textStyle="bold"
42         android:layout_below="@id/name"
43         android:text="Title" />
44
45     <TextView
46         android:id="@+id/description"
47         android:layout_width="wrap_content"
48         android:layout_height="wrap_content"
49         android:layout_centerHorizontal="true"
50         android:layout_marginTop="10dp"
51         android:textSize="14sp"
52         android:layout_below="@id/title"
53         android:textAlignment="center"
54         android:text="Description" />
55 </RelativeLayout>
56
57 <com.google.android.material.floatingactionbutton.FloatingActionButton
58     android:id="@+id/backButton"
59     android:layout_width="58dp"
60     android:layout_height="58dp"
61     android:layout_marginBottom="38dp"
62     android:clickable="true"
63     app:fabSize="normal"
64     app:layout_constraintBottom_toBottomOf="parent"
65     app:layout_constraintEnd_toEndOf="parent"
66     app:tint="@color/white"
67     android:backgroundTint="@color/black"
68     app:srcCompat="@drawable/baseline_arrow_back_24" />
69
70 </androidx.constraintlayout.widget.ConstraintLayout>

```

Nous avons une `ConstraintLayout` qui contient deux éléments:

- Une `RelativeLayout` qui contient quatres éléments:
 - Une `ImageView` : affiche une image correspondante un personnage de manga.
 - Une `TextView` : affiche le nom du personnage
 - Une `TextView` : affiche le titre du personnage
 - Une `TextView` : affiche la description du personnage
- Un `FloatingActionButton` : B=bouton flottant, souvent utilisé pour une action principale, ici pour revenir à l'activité précédente.

4. Mettre à jour l'activity:

```

1 import android.os.Bundle;

```



```

2  import android.view.View;
3  import android.widget.ImageView;
4  import android.widget.TextView;
5
6  import androidx.appcompat.app.AppCompatActivity;
7
8  import com.google.android.material.floatingactionbutton.FloatingActionButton;
9
10 public class MangaCharacterActivity extends AppCompatActivity {
11     ImageView pic;
12     TextView name;
13     TextView title;
14     TextView description;
15
16     FloatingActionButton backButton;
17
18     @Override
19     protected void onCreate(Bundle savedInstanceState) {
20         super.onCreate(savedInstanceState);
21         setContentView(R.layout.activity_manga_character);
22
23         pic = findViewById(R.id.pic);
24         name = findViewById(R.id.name);
25         title = findViewById(R.id.title);
26         description = findViewById(R.id.description);
27         backButton = findViewById(R.id.backButton);
28
29         Bundle extras = getIntent().getExtras();
30
31         if(extras != null) {
32             MangaCharacter character = (MangaCharacter) extras.get("data");
33
34             if(character != null) {
35                 int imageResId = this.getResources().getIdentifier(character.getImage(), "drawable", this.getPackageName());
36                 pic.setImageResource(imageResId);
37                 name.setText(character.getName());
38                 title.setText(character.getTitle());
39                 description.setText(character.getDescription());
40             }
41         }
42
43         backButton.setOnClickListener(new View.OnClickListener() {
44             @Override
45             public void onClick(View v) {
46                 finish();
47             }
48         });
49     }
50 }

```

- `pic`, `name`, `title`, `description` : Variables pour les composants `ImageView` et `TextView` qui afficheront les données du personnage.
- `backButton` : Un bouton flottant utilisé pour fermer l'activité.
- Au niveau de la méthode `onCreate` :
 - `Bundle extras = getIntent().getExtras();` : Récupère les données passées à cette activité.
 - Condition : Vérifie si des données supplémentaires ont été envoyées à l'activité. Si oui, un objet `MangaCharacter` est extrait et utilisé pour peupler les vues :

- Images : Récupère l'identifiant de ressource pour l'image associée au personnage et l'affiche dans `pic`.
- Textes : Met à jour les textes des `TextView` (`name`, `title`, `description`) avec les informations du personnage.
- Gestionnaire d'événements pour `backButton`
 - `setOnClickListener` : Ajoute un écouteur d'événements sur le `backButton` pour gérer les clics sur ce bouton.
 - `finish()` : Termine l'activité et retourne à l'activité précédente quand le bouton est cliqué.

5. Mettre à jour la classe `MainActivity` pour créer la navigation vers l'activité `MangaCharacterActivity` lorsqu'on clique sur un élément de la liste

```
1  ...
2
3  public class MainActivity extends AppCompatActivity {
4
5      ...
6
7      @Override
8      protected void onCreate(Bundle savedInstanceState) {
9          ...
10
11         listView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
12             @Override
13             public void onItemClick(AdapterView<?> parent, View view, int position, long id) {
14                 Intent intent = new Intent(view.getContext(), MangaCharacterActivity.class);
15                 intent.putExtra("data", characters[position]);
16                 startActivity(intent);
17             }
18         });
19
20     }
21 }
```

 Vous pourrez trouver le code complet sur ce lien [Github](#).